

Computação Gráfica e Processamento de Imagem



Professor: EDILSON LIMA

ATIVIDADE PRÁTICA – Visualização em Python

PARTE I

Atividade prática em Python para Google Colab baseada nos conceitos da sua aula de *Visualização (iluminação, tonalização e texturas)*.

A ideia é implementar um modelo simplificado de iluminação de Phong e aplicar tonalização em uma malha poligonal. Assim, os alunos podem visualizar como os parâmetros (difusa, especular e ambiente) afetam a renderização.

Atividade em Google Colab

```
# Atividade: Modelo de Iluminação de Phong em Python
# Disciplina: Computação Gráfica
# Professor: Edilson Lima
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Função para calcular iluminação de Phong
def phong_model(normal, light_dir, view_dir,
                la=0.1, ld=0.7, ls=0.5,
                ka=0.5, kd=0.7, ks=0.9, shininess=20):
    """
```

```
    normal: vetor normal da superfície
    light_dir: direção da luz
    view_dir: direção do observador
    la, ld, ls: intensidades ambiente, difusa e especular
    ka, kd, ks: coeficientes ambiente, difuso e especular
```

```

shininess: expoente especular
"""

# Normalização dos vetores
normal = normal / np.linalg.norm(normal)
light_dir = light_dir / np.linalg.norm(light_dir)
view_dir = view_dir / np.linalg.norm(view_dir)

# Componente ambiente
lamb = la * ka

# Componente difusa
diff = max(np.dot(normal, light_dir), 0)
ldiff = ld * kd * diff

# Componente especular
reflect_dir = 2 * np.dot(normal, light_dir) * normal - light_dir
spec = max(np.dot(view_dir, reflect_dir), 0) ** shininess
lspec = ls * ks * spec

# Intensidade final
return lamb + ldiff + lspec

# Criando uma grade de pontos simulando uma superfície
x = np.linspace(-1, 1, 50)
y = np.linspace(-1, 1, 50)
X, Y = np.meshgrid(x, y)

# Normal da superfície (plana)
normal = np.array([0, 0, 1])
light_dir = np.array([1, 1, 1])
view_dir = np.array([0, 0, 1])

# Calculando intensidade para cada ponto
Z = np.zeros_like(X)
intensity = np.zeros_like(X)

for i in range(X.shape[0]):
    for j in range(X.shape[1]):
        intensity[i, j] = phong_model(normal, light_dir, view_dir)

# Plotando resultado
plt.figure(figsize=(6,6))
plt.imshow(intensity, cmap='inferno', extent=(-1,1,-1,1))

```

```
plt.title("Iluminação de Phong - Superfície Plana")  
plt.colorbar(label="Intensidade da luz")  
plt.show()
```

Objetivos da atividade

- Entender como os três componentes do modelo de Phong (ambiente, difusa e especular) influenciam a iluminação.
- Explorar os parâmetros k_a , k_d , k_s e shininess para observar diferentes efeitos visuais.
- Relacionar teoria (aula) com prática (código).

Sugestões:

1. Alterar os valores de shininess e observar como o brilho especular muda.
2. Modificar a direção da luz (`light_dir`) e verificar como a iluminação se distribui.
3. Implementar tonalização constante vs interpolada em uma malha simples.
4. Adicionar uma textura (imagem) sobre a superfície para simular materiais reais.

PARTE II

Utilizar Google Colab baseada nos conceitos da aula de *Visualização* (iluminação, tonalização e texturas). A ideia é que possam experimentar o modelo de iluminação de Phong e comparar tonalização constante e interpolada em um exemplo simples.

Atividade em Google Colab

Objetivo

Explorar os conceitos de iluminação local (Phong) e tonalização (shading) aplicados em computação gráfica, simulando reflexões difusa, especular e ambiente em polígonos.

Passo 1 – Configuração inicial

```
import numpy as np
import matplotlib.pyplot as plt
```

Passo 2 – Funções de iluminação (Modelo de Phong simplificado)

```
def phong_reflection(normal, light_dir, view_dir,
                    ka=0.2, kd=0.7, ks=0.5, s=10,
                    la=np.array([0.1,0.1,0.1]),
                    ld=np.array([1.0,1.0,1.0]),
                    ls=np.array([1.0,1.0,1.0])):
    """
    Calcula a cor resultante em um ponto usando o modelo de Phong.
    """
    # Normalização
    normal = normal / np.linalg.norm(normal)
    light_dir = light_dir / np.linalg.norm(light_dir)
    view_dir = view_dir / np.linalg.norm(view_dir)

    # Ambiente
    lamb = ka * la

    # Difusa
    diff = max(np.dot(normal, light_dir), 0.0)
    ldiff = kd * diff * ld

    # Especular
    reflect_dir = 2 * np.dot(normal, light_dir) * normal - light_dir
    spec = max(np.dot(view_dir, reflect_dir), 0.0) ** s
    lspec = ks * spec * ls
```

```
return lamb + Idiff + Ispec
```

Passo 3 – Comparando tonalização constante vs interpolada

```
# Definição de vértices de um triângulo
vertices = np.array([[0,0,0],
                    [1,0,0],
                    [0.5,1,0]])

# Normais (iguais para simplificação)
normal = np.array([0,0,1])

# Direções de luz e câmera
light_dir = np.array([0.5,0.5,1])
view_dir = np.array([0,0,1])

# Cores nos vértices (Phong)
colors = [phong_reflection(normal, light_dir, view_dir) for v in vertices]

# Tonalização constante: mesma cor em todo polígono
constant_color = np.mean(colors, axis=0)

# Plot
fig, ax = plt.subplots(1,2, figsize=(8,4))

# Constante
ax[0].fill(vertices[:,0], vertices[:,1], color=constant_color)
ax[0].set_title("Tonalização Constante")

# Interpolada (usando Gouraud-like shading)
tri = plt.tripcolor(vertices[:,0], vertices[:,1], [0,1,2], facecolors=colors,
                    shading='gouraud')
ax[1].set_title("Tonalização Interpolada")

plt.show()
```

Passo 4 – Desafio

1. Alterar os parâmetros k_a , k_d , k_s e s e observar como a iluminação muda.
2. Testar diferentes direções de luz (`light_dir`).
3. Criar novos polígonos e comparar tonalização constante vs interpolada.
4. Adicionar uma textura simples (imagem 2D) sobre o polígono usando `imshow` e coordenadas.

Essa atividade conecta diretamente os conceitos da aula: iluminação local (Phong), difusa/especular/ambiente, e tonalização constante vs interpolada.